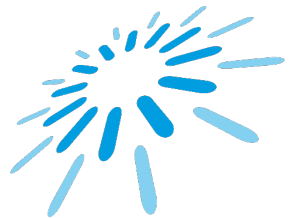




Software Architecture and Design Patterns

Prof. Dr. Jóakim von Kistowski

Lecture 03



TH Aschaffenburg
university of applied sciences

	Monday	Tuesday	Wednesday	Thursday	Friday
08:00 - 09:30					Vorlesung Software Design
09:45 - 11:15					Übung Software Design
11:30 - 13:00			Lecture Software Design International		
	Lunch Break				
14:00 - 15:30			Exercises Software Design International		
15:45 - 17:15					
17:30 - 19:00					

What is Software Quality?

What is a Software Quality Scenario?

Software Architecture: Design Decisions



ASRs – Archit. Significant Requirements

ADRs – Architectural Decision Records

Utility Trees

Architecture Significant Requirements (ASRs)

ASRs are those requirements that have a measurable impact on a software system's architecture.

"ASR Self-Check"

Which of the following are ASRs?

- "Temperature should be displayed in Celsius not Fahrenheit on the webpages"
- "The system should provide 99.999 % availability"
- "System x should provide an interface for system y to submit requests"
- "A dropdown list of cities should be provided to help user specify address"

ASRs Indicators & Heuristics (1/2)

ASRs indicators for architects

- cross-cutting, system wide impact (security, monitoring), targeting trade-offs
- business value and risk
- key stakeholder concern
- unusual QoS requirement (order of magnitude more advanced)
- Strictness (constraining, limiting, non-negotiable)
- external dependencies (APIs) that are uncontrollable, unpredictable or unreliable,
- Assumption breaking and difficult to achieve

ASRs Indicators & Heuristics (2/2)

ASRs heuristics for requirements engineer

- Quality attributes
- Core features
- Constraints
- Application Environment

[collected from Peter Eeles, Olaf Zimmermann]
Read more: Muhammad Ali Babar, Lianping Chen, Bashar Nuseibeh, "Characterizing Architecturally Significant Requirements", IEEE Software vol. 30, p. 38-45, March-April, 2013

Design Decision Category	Look for Requirements Addressing . . .
Allocation of Responsibilities	Planned evolution of responsibilities, user roles, system modes, major processing steps, commercial packages
Coordination Model	Properties of the coordination (timeliness, currency, completeness, correctness, and consistency) Names of external elements, protocols, sensors or actuators (devices), middleware, network configurations (including their security properties) Evolution requirements on the list above
Data Model	Processing steps, information flows, major domain entities, access rights, persistence, evolution requirements
Management of Resources	Time, concurrency, memory footprint, scheduling, multiple users, multiple activities, devices, energy usage, soft resources (buffers, queues, etc.) Scalability requirements on the list above
Mapping among Architectural Elements	Plans for teaming, processors, families of processors, evolution of processors, network configurations
Binding Time Decisions	Extension of or flexibility of functionality, regional distinctions, language distinctions, portability, calibrations, configurations
Choice of Technology	Named technologies, changes to technologies (planned and unplanned)

Which ASRs to gather from a requirements document

Software Architecture: Design Decisions



ASRs – Archit. Significant Requirements

ADRs – Architectural Decision Records

Utility Trees

Software Architecture Methods

- Goal: Make Decisions or evaluate software architecture (**alternatives**)
- Focus on different aspects
 - Requirements
 - Quality-Requirements (Non-Functional Requirements)
 - Technical Framework
 - Organizational Framework
- May differ in granularity and detail, with which the different aspects are taken into consideration

How do we make a final architectural decision?

Example Decision:

“Create Domain Specific Language for Domain Experts”

Alternative 1: Domain expert writes a requirements document / domain concept and technical developer codes this down in a native programming language (e.g., Java/C#)

Alternative 2: We design a specific language for our domain experts. They can use this language to document their requirements in a machine-readable way. We can then generate native code from this specification without manual coding effort by technical developers.

How do we reach a decision?

What information do we need to be able to reach a decision?

Architecture Decision Record (ADR)

Documents an architectural design decision.

- Precondition
 - A solvable problem
 - Existing decisions and surrounding conditions / frameworks / constraints
- Approach
 - Name problem and constraints explicitly
 - Create different alternative solutions
 - Estimate and evaluate effects of the different solution alternatives
 - Focus: Different quality attributes

Task – Prepare Decision

Decision: Create Domain Specific Language for Domain Experts

Problem: Domain experts have expensive knowledge about the Domain but require technical experts to put this knowledge into source code. The knowledge transfer process is error prone.

Constraints: We have domain experts who don't know how to code and we have technical experts who have limited knowledge about the domain.

Solution Alternative 1: Domain expert writes a requirements document / domain concept and technical developer codes this down in a native programming language (e.g., Java/C#)

Solution Alternative 2: We design a specific language for our domain experts. They can use this language to document their requirements in a machine-readable way. We can then generate native code from this specification without manual coding effort by technical developers.

What's missing in order to evaluate the alternatives?

Task – Evaluate Alternatives

Decision: Create Domain Specific Language for Domain Experts

Solution Alternative 1: Domain expert writes a requirements document / domain concept and technical developer codes this down in a native programming language (e.g., Java/C#)

Solution Alternative 2: We design a specific language for our domain experts. They can use this language to document their requirements in a machine-readable way. We can then generate native code from this specification without manual coding effort by technical developers.

Quality Attribute	Domain Concept + native Code	Specific Language
Initial Time-to-Market		
Maintainability		
Portability (Technology-Shift)		
Performance		

Task – Example Evaluation

Decision: Create Domain Specific Language for Domain Experts

Solution Alternative 1: Domain expert writes a requirements document / domain concept and technical developer codes this down in a native programming language (e.g., Java/C#)

Solution Alternative 2: We design a specific language for our domain experts. They can use this language to document their requirements in a machine-readable way. We can then generate native code from this specification without manual coding effort by technical developers.

Quality Attribute	Domain Concept + native Code	Specific Language
Initial Time-to-Market	0	--
Maintainability	--	+
Portability (Technology-Shift)	--	+
Performance	++	--

ADR – Structure (1/2)

According to Arc42/Nygard:

Section	Description
Title	Decisions should have a proper name. For example, “ADR 42: Strategy-Pattern for multiple checking algorithms” or “ADR 9: Keycloak for secrets management”
Context	Describe the situation, including technical, political, social, and project aspects. These forces might be in tension.
Decision	What is our decision (reply to the forces described in the context)
Status	A decision may be “proposed” if the important stakeholders haven’t agreed with it yet, or “accepted” once it is agreed. It could also be “deprecated” or “superseded”, hopefully with a reference to its replacement.
Consequences	What will or can happen as consequences of this decision. All consequences should be listed here, not just the “positive” ones. A particular decision may have positive, negative, and neutral consequences, but all of them affect the team and project in the future.

ADR – Final Decision

Decide based on:

- Quality evaluation result and importance/ranking of quality attributes.
- Costs
- Risks
- When in doubt: Talk with your stakeholders.

No Silver Bullet

No Alternative is ALWAYS the best option.

Everything depends on context and the your project's (non-functional) requirements.

Decisions must be weighed and ranked against specific project requirements.

A general pro/con list is not an architectural decision!

ADR – Structure (2/2)

Modified Arc42/Nygard:

Section	Important Aspects
Title	
Context	Problem Constraints Quality Objectives
Alternatives	Your Alternatives with advantages and disadvantages
Decision	Evaluation Final Decision
Status	“proposed”, “accepted”, “deprecated”, “superseded”, ...
Consequences	Advantages Disadvantages Risks

Task – Decide!

Pick a Game-Engine for our online shooter! Write a (very) short ADR.

Pick to game engines you know (or have heard of) and use them as alternatives for your decision.

Online-Shooter

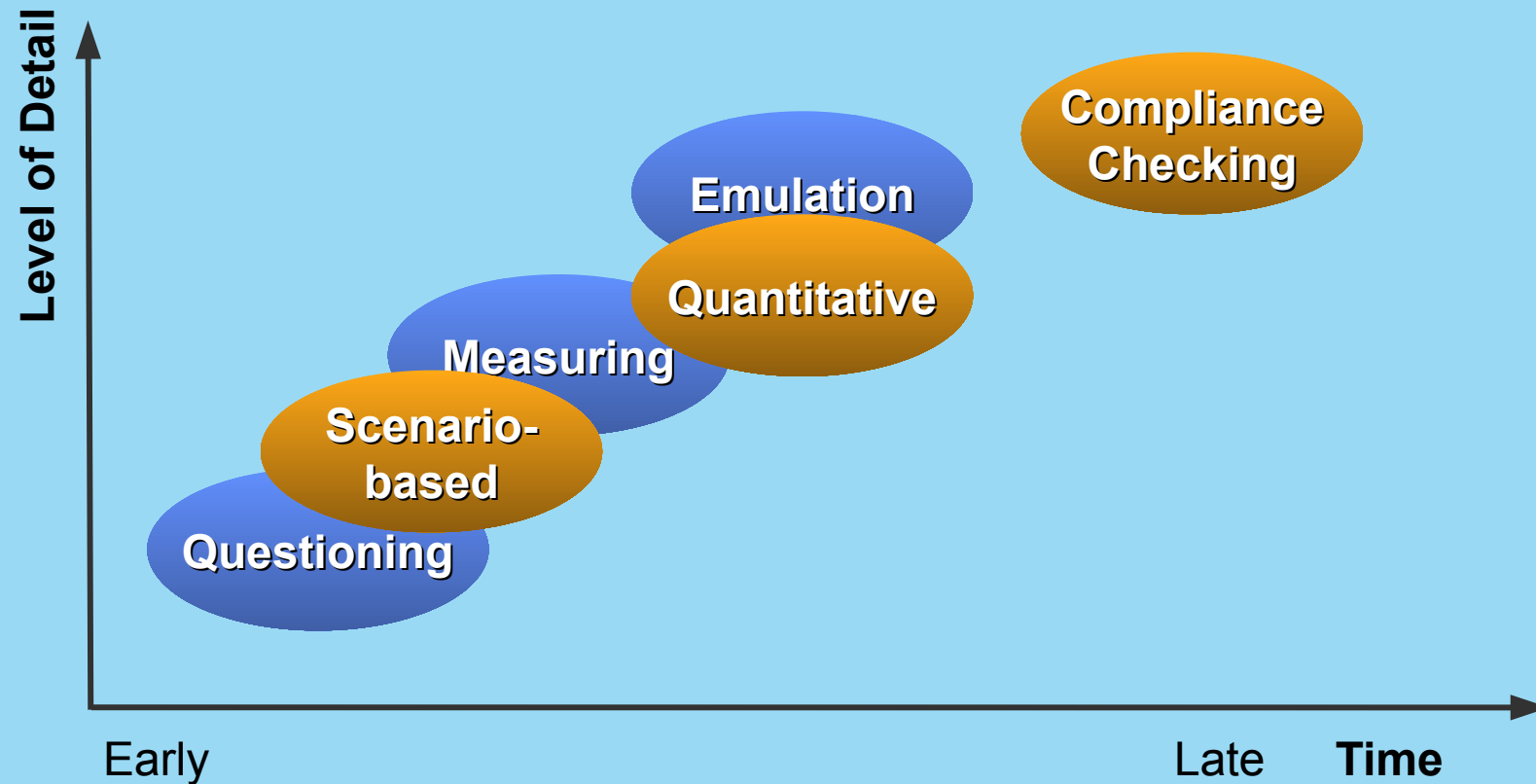


Structure

Section	Important Aspects
Title	
Context	Problem Constraints Quality Objectives
Alternatives	Your Alternatives with advantages and disadvantages
Decision	Evaluation Final Decision
Status	We assume the decision is final
Consequences	Advantages Disadvantages Risks

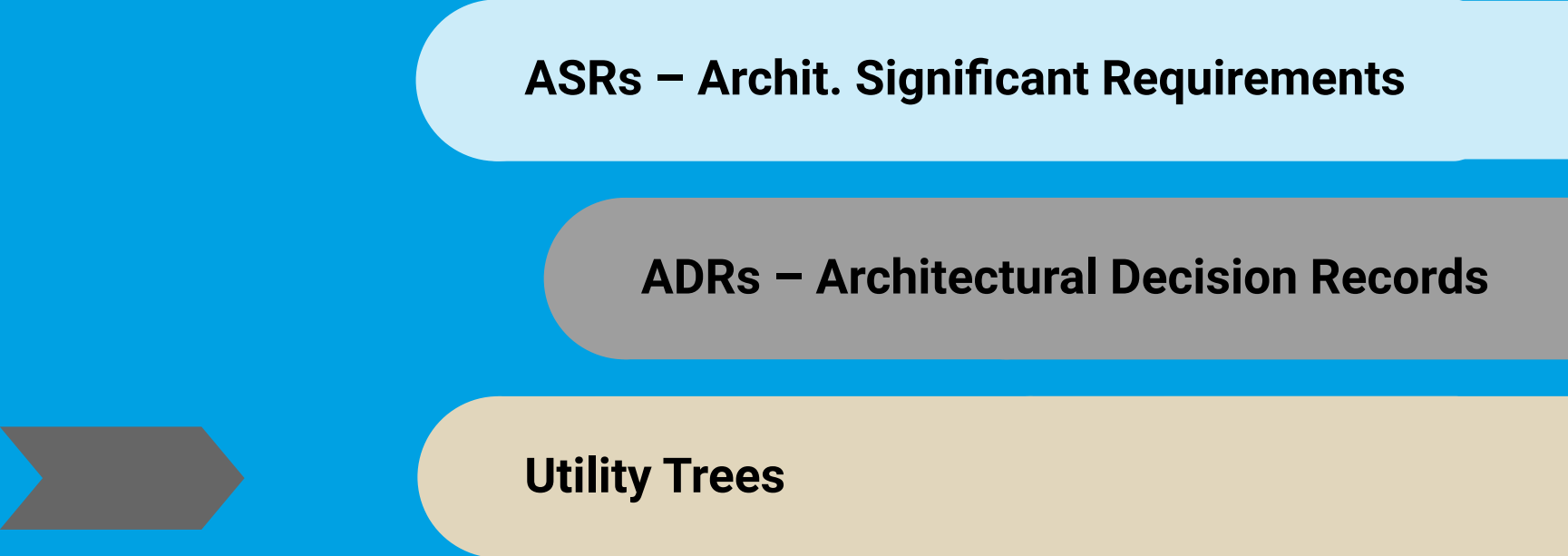
What role do quality scenarios play when making architectural decisions?

Recap: Architecture Evaluation Techniques



Technique depends on granularity of decision.

Software Architecture: Design Decisions



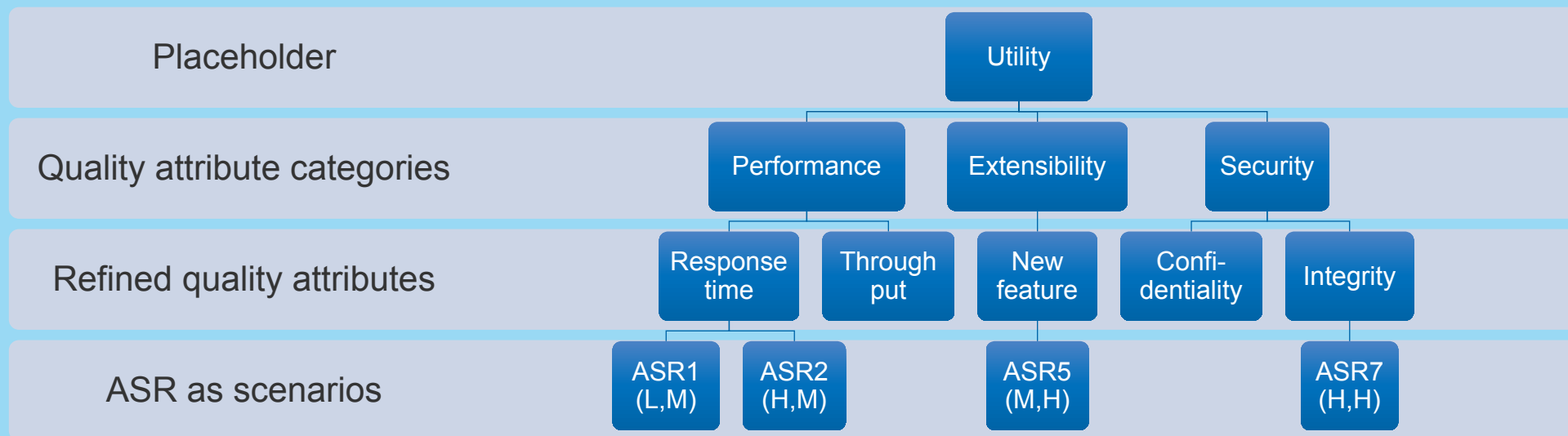
ASRs – Archit. Significant Requirements

ADRs – Architectural Decision Records

Utility Trees

Utility Tree

- All ASRs covered



- Priority of ASR with two values {High, Medium, Low} i.e. (M,L)
 - Impact on architecture
 - Business or mission value

Utility Tree Example (Excerpt)

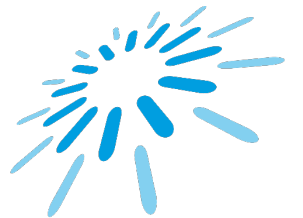
	Quality Attribute	Attribute Refinement	ASR
Utility	Performance	Transaction response time	A user updates a patient's account in response to a change-of-address notification while the system is under peak load, and the transaction completes in less than 0.75 second. (H,M) A user updates a patient's account in response to a change-of-address notification while the system is under double the peak load, and the transaction completes in less than 4 seconds. (L,M)
		Throughput	At peak load, the system is able to complete 150 normalized transactions per second. (M,M)
	Usability	Proficiency training	A new hire with two or more years' experience in the business becomes proficient in Nightingale's core functions in less than 1 week. (M,L) A user in a particular context asks for help, and the system provides help for that context, within 3 seconds. (H,M)
		Normal operations	A hospital payment officer initiates a payment plan for a patient while interacting with that patient and completes the process without the system introducing delays. (M,M)
	Configurability	User-defined changes	A hospital increases the fee for a particular service. The configuration team makes the change in 1 working day; no source code needs to change. (H,L)
	Maintainability	Routine changes	A maintainer encounters search- and response-time deficiencies, fixes the bug, and distributes the bug fix with no more than 3 person-days of effort. (H,M) A reporting requirement requires a change to the report-generating metadata. Change is made in 4 person-hours of effort. (M,L)
		Upgrades to commercial	The database vendor releases a new version that must be





Until next time!

Software Architecture and Design Patterns
Prof. Dr. Jóakim von Kistowski



TH Aschaffenburg
university of applied sciences